

DL 101: Basic introduction to deep learning with its application in biomedical related fields

Tianyu Zhan 

Data and Statistical Sciences, AbbVie Inc.,
North Chicago, Illinois, USA

Correspondence

Tianyu Zhan, Data and Statistical
Sciences, AbbVie Inc., 1 Waukegan Rd,
North Chicago, IL 60064, USA.
Email: tianyu.zhan.stats@gmail.com

Deep learning is a subfield of machine learning used to learn representations of data by successive layers. Remarkable achievements and breakthroughs have been made in image classification, speech recognition, et cetera, but the full capability of deep learning is still under exploration. As statistical researchers and practitioners, we are especially interested in leveraging and advancing deep learning techniques to address important and impactful problems in biomedical and other related fields. In this article, we provide a basic introduction to Feed-forward Neural Networks (FNN) along with some intuitive explanations behind its strong functional representation. Guidance is provided on how to choose quite a few hyperparameters in neural networks for a specific problem. We further discuss several more advanced frameworks in deep learning. Some successful applications of deep learning in biomedical fields are also demonstrated. With this beginner's guide, we hope that interested readers can include deep learning in their toolbox to tackle future real-world questions and challenges.

KEYWORDS

feedforward neural networks, machine learning, representation learning

1 | INTRODUCTION

Although deep learning is a fairly old subfield of machine learning, it rose to prominence in the past few years due to advances in computation.¹ Many breakthroughs have been made by deep learning in historically difficult areas in machine learning,^{1,2} such as image classification/transcription,³⁻⁶ speech recognition,⁷⁻⁹ natural language processing¹⁰⁻¹² and autonomous driving.¹³⁻¹⁵ Several intrinsic properties make deep learning a feasible and popular option in the artificial intelligence (AI) revolution: simplicity, scalability and versatility.¹ Traditional machine learning methods usually require feature engineering to manually find proper initial input data before model fitting, while deep learning automates this step.¹ Relevant and important features can be automatically extracted by multiple layers. With the ability of conducting training on small batches of data, deep learning is able to accommodate big and complex datasets. Parallel computing is also available to boost computational efficiency. Since training algorithms are usually conducted in an iterative fashion, deep learning models can be continuously trained with additional data.¹ Reproducing results is possible with well-trained models saved in files. With these appealing features, deep learning can be a useful and powerful option to address important and complex biomedical problems.

The goal of our article is to provide a basic and general introduction to deep learning, such that interested readers may explore other more advanced or specialized deep learning techniques to address specific problems. In Section 2, we introduce the general ideas behind deep learning with a simple subtype called feedforward neural networks (FNN). Training process as well as some hyperparameters are also discussed. A simulation study is conducted in Section 3 to

demonstrate the strong functional representation of deep learning to approximate nonlinear functions, and the procedure of how to fine-tune hyperparameters. Section 4 talks about some variants of FNN, such as Convolutional neural networks (CNN), Recurrent Neural Networks (RNN), autoencoders and generative adversarial networks (GAN). In Section 5, we highlight some selective applications of deep learning in medical image analyses, complex innovative clinical trial designs and omics studies. Concluding remarks are provided in Section 6.

2 | FEEDFORWARD NEURAL NETWORKS

Among many branches of deep learning, we utilize the so-called feedforward neural networks (FNN) as a starting point in this section to discuss the general spirit and idea behind this powerful subfield of machine learning.

2.1 | Function approximation

Suppose that f^* is some function that maps input $\mathbf{x}$ to output y . The goal of FNN is to construct a mapping function $y = f(\mathbf{x}; \boldsymbol{\phi})$ with the parameters $\boldsymbol{\phi}$ that have the best function approximation of f^* .² For demonstration purposes, we consider that y is a numerical value and $\mathbf{x}$ has a dimension of p . Here, we consider $\mathbf{x}$ as a column vector for one replicate of input data. In the illustrating example of Figure 1, this FNN takes $\mathbf{x}$ with $p = 3$ as input on the left to output y on the right via four hidden layers. FNN $f(\mathbf{x}; \boldsymbol{\phi})$ is formulated by successively aggregating multiple layers of representations,

$$f(\mathbf{x}; \boldsymbol{\phi}) = f^{(5)}(f^{(4)}(f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x}))))), \quad (1)$$

$$f^{(h)}(\mathbf{z}) = g_h(\mathbf{w}_h^T \mathbf{z} + \mathbf{c}_h), \quad (2)$$

where $f^{(1)}$ is the function connecting the input layer to the first hidden layer, ..., and $f^{(5)}$ connects the fourth hidden layer to the output layer. Following notations in Goodfellow et al.,² we call $\mathbf{w}_h$ in (2) the weights, $\mathbf{c}_h$ the biases, and $g_h(\cdot)$ the activation function to be discussed next. For example in $f^{(1)}$, $\mathbf{w}_1$ of a dimension 3×5 and $\mathbf{c}_1$ of 5×1 are applied to input data $\mathbf{x}$ of a dimension 3×1 to match the input dimension of 5 in the first hidden layer. Essentially, the output from a previous layer is utilized as the input for the next layer to formulate such multilayer networks. In the FNN $y = f(\mathbf{x}; \boldsymbol{\phi})$, the parameter $\boldsymbol{\phi}$ is therefore a stack of all weights and biases $\boldsymbol{\phi} = \{\mathbf{w}_h, \mathbf{c}_h, h \in \{1, 2, 3, 4, 5\}\}$.

The inner activation functions (ie, g_1, g_2, g_3 , and g_4 in (2)) are usually chosen as some nonlinear function to be applied element-wise.² A popular choice is the rectified linear unit (ReLU) function defined as $g(z) = \max(0, z)$,^{16,17} because it preserves many of the properties that make linear models easy to optimize with gradient-based methods.² The sigmoid function $g(z) = 1/(1 + \exp(-z))$ ¹⁸ is another common inner activation function, but it saturates to either 0 or 1 when z is extreme.^{1,2} It can be helpful to standardize each feature to have a mean of zero and a unit standard deviation before training.¹ The last-layer activation function (ie, g_5) is usually chosen as the sigmoid or the softmax function (a generalization of the sigmoid function to accommodate a discrete variable with multiple values) for binary and multiclass

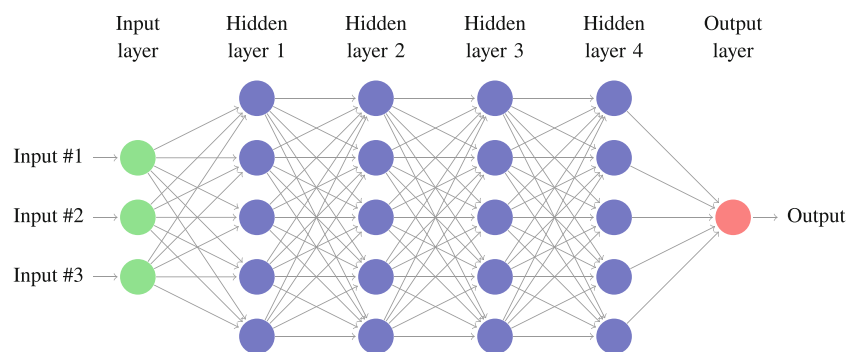


FIGURE 1 An FNN with four hidden layers

classification problems, while it is set as the identity function if the output y is continuous. The estimated or predictive output can also be truncated if y has certain constraints.

2.2 | Learning the XOR function

In this section, we consider a toy example of XOR function (“exclusive or” function) presented in Goodfellow et al² to illustrate the ability of FNN to approximate nonlinear functions. XOR returns 1 if only one input is equal to 1, and returns 0 otherwise (Figure 2A). Consider a linear model defined as

$$f_1(\mathbf{x}; \boldsymbol{\alpha}, \beta) = \boldsymbol{\alpha}^T \mathbf{x} + \beta, \quad (3)$$

where $\mathbf{x} = (x_1, x_2)^T$ is the two-dimensional input vector. By minimizing the MSE loss function, one obtains $\boldsymbol{\alpha} = (0, 0)^T$ and $\beta = 1/2$.² A linear model cannot be directly applied to this problem, because it returns the same output of $1/2$ for all input data points. An intuitive explanation is that the linear model in (3) has fixed coefficients $\boldsymbol{\alpha}$. However, based on Figure 2A, the coefficient of x_2 needs to be positive when $x_1 = 0$, while it has to be negative when $x_1 = 1$. In this example, we only evaluate training error without considering testing error.

As an alternative, we can utilize an FNN with one hidden layer and two nodes to perfectly fit the data,

$$f(\mathbf{x}; \mathbf{w}_1, \mathbf{w}_2) = \mathbf{w}_2^T \text{ReLU}(\mathbf{w}_1^T \mathbf{x}). \quad (4)$$

This is a simplified version of FNN in (1) with zero bias parameters c_1 and c_2 . The inner activation function g_1 is ReLU, and the outer activation function g_2 is linear. We define $\mathbf{X}$ as a 2 by 4 matrix, where each column corresponds to each of the four training inputs in Figure 2A,

$$\mathbf{X} = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}.$$

The column vector $\mathbf{x}$ in (3) represents a single replicate of data, while the matrix $\mathbf{X}$ is a stack matrix with multiple replicates. The weight parameters $\mathbf{w}_1$ and $\mathbf{w}_2$ in FNN $f(\mathbf{X}; \mathbf{w}_1, \mathbf{w}_2)$ are set as,

$$\mathbf{w}_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad \mathbf{w}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

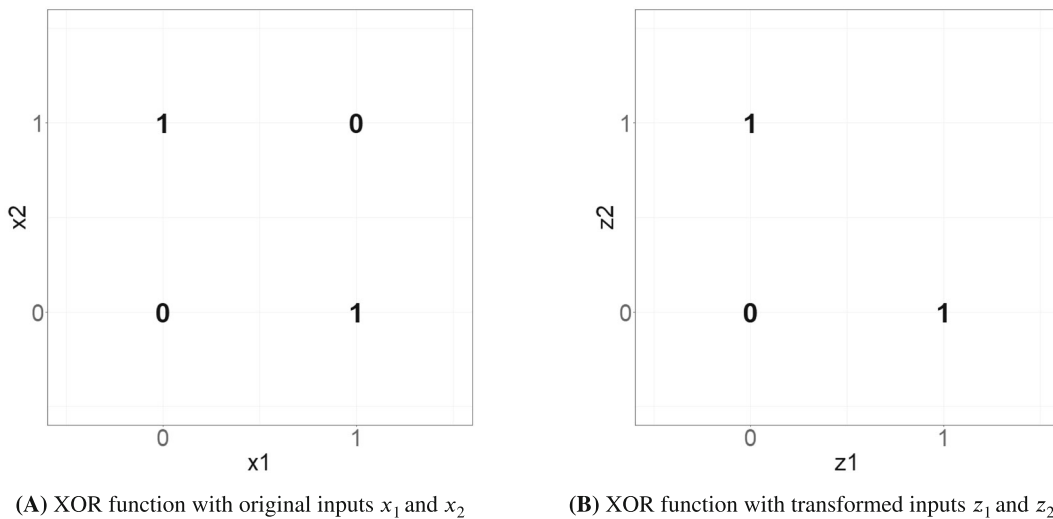


FIGURE 2 A linear model cannot correctly classify XOR function in original inputs (2a), but can solve the problem with transformed inputs (2b)

One can further compute $\mathbf{w}_1^T \mathbf{X}$, $\mathbf{Z} = \text{ReLU}(\mathbf{w}_1^T \mathbf{X})$ and $f(\mathbf{X}; \mathbf{w}_1, \mathbf{w}_2) = \mathbf{w}_2^T \mathbf{Z}$ as the following,

$$\mathbf{w}_1^T \mathbf{X} = \begin{bmatrix} 0 & -1 & 1 & 0 \\ 0 & 1 & -1 & 0 \end{bmatrix} \quad \mathbf{Z} = \text{ReLU}(\mathbf{w}_1^T \mathbf{X}) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad f(\mathbf{X}; \mathbf{w}_1, \mathbf{w}_2) = \mathbf{w}_2^T \mathbf{Z} = \begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix}.$$

As shown in Figure 2B, the ReLU activation function and $\mathbf{w}_1$ transform original input $\mathbf{X}$ to $\mathbf{Z}$. This problem can now be solved by a linear model because the first and the fourth column in $\mathbf{Z}$ are mapped to a single point (0, 0) in Figure 2B, and are predicted as 0 by $f(\mathbf{X}; \mathbf{w}_1, \mathbf{w}_2)$. This is also related to the idea of support vector machine (SVM)¹⁹ by mapping data to hyperplanes with kernel functions. Following this simple illustration, one can further build a deeper and wider FNN with more layers and nodes to approximate more complicated functions. Those multiple hidden layers and associated nodes contribute to the strong functional representation of neural networks.

However, there are several challenges to apply FNN in practice. Although there may exist an FNN that can well approximate the objective function as discussed in Section 2.4.1, those theorems do not explicitly specify the values of ϕ in $f(\mathbf{x}; \phi)$. We still need to learn such FNN or construct one close to the true model. The identifiability of ϕ in FNN is an ongoing research topic. On the example of XOR, one can find another set of ϕ leading to the same predictors, for example the weight and bias parameters provided in section 6.1 in Goodfellow et al.² Since the loss function to estimate ϕ in FNN is usually non-convex, we need an efficient algorithm to feasibly estimate ϕ based on finite training data and to achieve satisfactory performance in the testing dataset. In the next section, we discuss how to estimate ϕ to obtain a learnable FNN.

2.3 | Training of Neural Networks

Typically, training an FNN involves the following four components: layers, input data and corresponding output, loss function, and optimizer.¹ Layers are combined in FNN to create neural networks as in Figure 1. Its specification is discussed next. Input data are denoted as $\{\mathbf{x}_i\}_{i=1}^n$, and outputs are $\{y_i\}_{i=1}^n$ with n as the training data size. The loss function or cost function measures how well the FNN $f(\mathbf{x}; \phi)$ approximates the output y (essentially the underlying function f^*). By minimizing this loss function in the training process, we obtain $\hat{\phi}$ as an estimator of ϕ and therefore the trained FNN $f(\mathbf{x}; \hat{\phi})$ becomes a functional approximation of f^* . One can choose mean squared error loss for continuous y ,

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \sum_{i=1}^n [y_i - f(\mathbf{x}_i; \phi)]^2, \quad (5)$$

or the cross entropy (ie, the negative log-likelihood) for binary classification,

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \sum_{i=1}^n -\{y_i \log [f(\mathbf{x}_i; \phi)] + [1 - y_i] \log [1 - f(\mathbf{x}_i; \phi)]\}. \quad (6)$$

There are several other choices for the loss function, such as mean absolute error and the cross entropy for a continuous variable. However, some metrics, for example, Area Under the ROC Curve, cannot be directly applied, because the loss function should be differentiable in order to calculate gradient in training and be computable given a mini-batch of data.¹ The optimizer determines how the network will be updated based on the loss function. A major challenge of optimizing loss functions in (5) or (6) to obtain $\hat{\phi}$ is that they are usually non-convex due to the nonlinearity of FNN in (1).² FNN is often trained by iterative, gradient-based optimizers to drive the cost function to a very low value, rather than common convex optimization with global convergence guarantees.² Some popular choices of algorithms are: AdaGrad,²⁰ RMSProp,²¹ and Adam.²²

2.4 | Hyperparameters

There are several hyperparameters involved in the structure and training of FNN. We introduce some common parameters in this section, and discuss how to choose them in practice in the next section.

2.4.1 | FNN structure

The structure or skeleton of FNN in Figure 1 is determined by the number of hidden layers and the number of nodes per layer. Based on the universal approximation theorem, a shallow FNN with one hidden layer with a sufficiently large number of nodes (usually in an order of exponential of input dimension p) and certain activation functions can approximate any continuous function up to a desired error tolerance.^{18,23} This theorem essentially says that for any continuous function f^* , there exists an FNN $f(\mathbf{x}; \boldsymbol{\phi})$ with one hidden layer in (1) that can represent it fairly well. In recent years, interest has shifted to deeper neural networks, which usually have a better generalization performance and require a reduced number of units.² A deeper structure is also consistent with the intuition behind the strong functional representation of FNN.

However, challenges still exist because we are not guaranteed to learn this underlying representation.² The trained FNN can be very different from the truth if the architecture is not properly chosen. Moreover, the optimization algorithm may fail to locate the optimal parameters. With limited training data, the fitted FNN may not perform well in validation tasks. Therefore, it is important to choose hyperparameters in FNN training by cross-validation to achieve satisfactory performance.

2.4.2 | Learning rate

Learning rate is a positive scalar determining the size of the step in the gradient-based optimizers to obtain $\hat{\boldsymbol{\phi}}$ in (5) or (6). This important hyperparameter controls the effective capacity of FNN, and should be set at a proper value.² Gradient descent can inadvertently increase rather than decrease the training error when the learning rate is too large; while training becomes slower and may get stuck in local minima when the learning rate is too small.² Several advanced optimization methods mentioned in Section 2.3 adapt their learning rates during training or leverage information contained in the second derivatives of the cost function.

2.4.3 | Batch size

Mini-batch stochastic gradient descent (mini-batch SGD) is a general class of algorithms which randomly sample a small number of examples from the datasets to compute the loss functions in (5) and (6), and which then take the average over those samples.² This approach is computationally more efficient than batch SGD, which uses all data to calculate gradient at every step,¹ and also offers a regulation effect by introducing certain noise.² A smaller batch size usually has a better generalization error, but may also require a smaller learning rate to maintain stability.²

2.4.4 | Number of epochs

Another related hyperparameter is the number of training epochs.² One epoch is usually defined as the process of training FNN with one round of gradient update over all the training data,¹ while iteration is a related notation on one batch of data with its size introduced in the section above. For an example with training data size of 60 000 and batch size of 128, there are 469 (the integer that 60 000/128 rounds up) iterations of gradient update performed for one epoch.¹

The training loss usually decreases with every epoch, but the validation loss may decrease and then increase to form an asymmetric U-shaped curve.^{1,2} Without closely monitoring the validation loss, one may overoptimize the training data and obtain a learning representation that lacks generalization ability.¹ In practice, we can utilize cross-validation to select a proper number of training epochs. Alternatively, early stopping can be implemented to terminate the training when the training or validation error does not improve after a certain number of iterations.²

2.4.5 | Dropout rate

Dropout is a regulation technique that randomly sets some layer outputs to be zero during the training process.²⁴ It is a powerful and inexpensive computation method to reduce testing error without increasing training error.² This technique

can be applied to each hidden layer, but usually not to the last output layer. The core idea is that introducing noise in the output values of a layer can break up happenstance patterns that are not significant.¹ The dropout rate is the fraction of features that are zeroed out, and is usually set less than 0.5.¹ Some other regulation approaches to prevent over-fitting are: L_1 and L_2 regulation, noise injection,²⁵ early stopping, parameter sharing²⁶ and bootstrap aggregating.²⁷

2.4.6 | Other qualitative hyperparameters

In addition to those common quantitative hyperparameters discussed above, there are also some qualitative parameters in FNN that can be chosen by cross-validation. For example, activation functions for hidden layers can be selected among ReLU,¹⁷ sigmoid,¹⁸ and tanh activation function $g(z) = \{\exp(z) - \exp(-z)\} / \{\exp(z) + \exp(-z)\}$.² The optimization algorithm for FNN training can be chosen within AdaGrad,²⁰ RMSProp,²¹ and Adam,²² et cetera.

3 | SIMULATION STUDY

In this section, we consider a benchmark task for neural networks to distinguish between two spirals²⁸ as visualized in Figure 3A. The goal of this binary classification problem is to determine if a point should be classified as a dot or a triangle

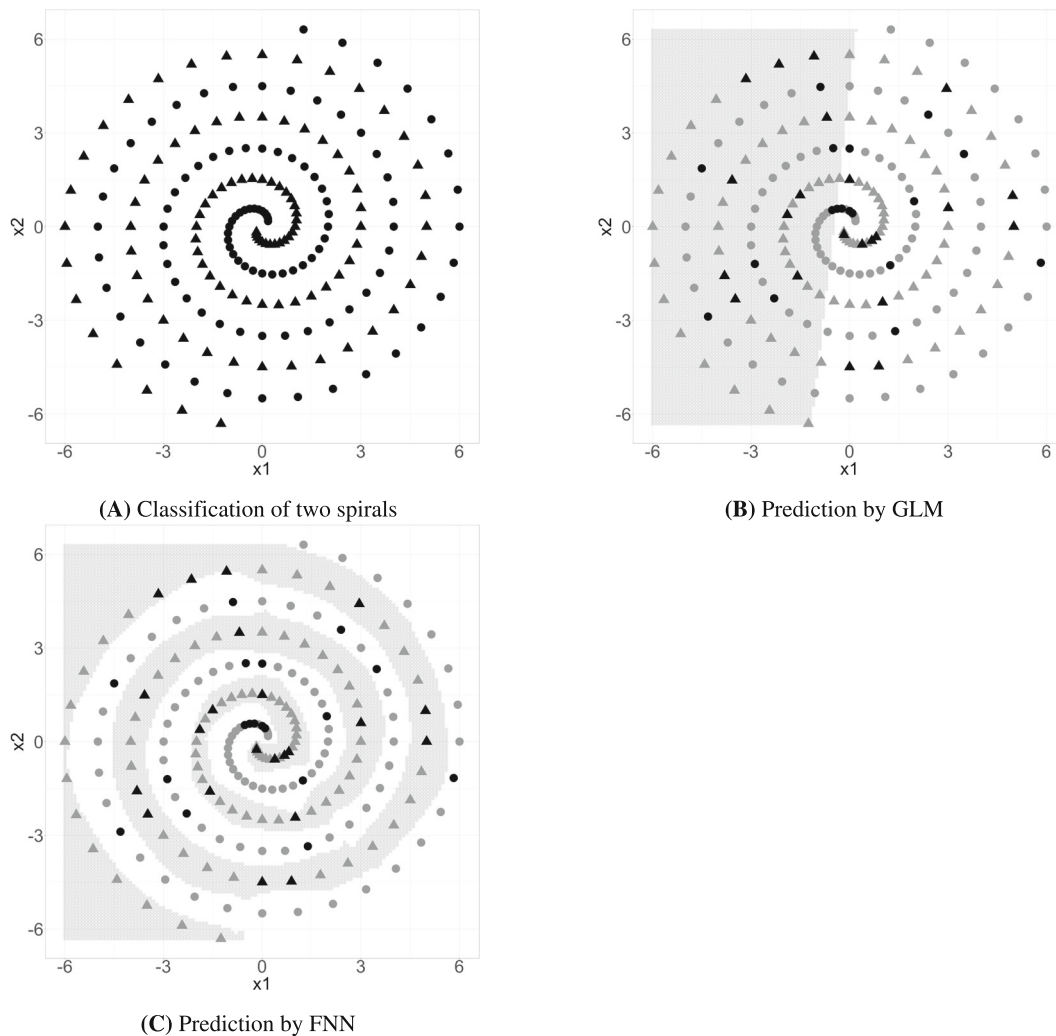


FIGURE 3 The goal of this binary classification problem in (A) is to determine if a point should be classified as a dot or a triangle based on two input units: x_1 and x_2 . The testing data (20%) are labeled with black dots or triangles and the training data (80%) are labeled with gray symbols in (B,C). The generalized linear model (GLM) in (B) and FNN in (C) predict triangles in shaded area, or dots in unshaded area

based on two input units (x_1 and x_2). The input data of FNN is considered as $\mathbf{x} = (x_1, x_2, x_1^2, x_2^2, x_1x_2)^T$ with squared and interaction terms. The same input data vector is used for the generalized linear model (GLM) discussed later. Although this task is easy to visualize, it is relatively hard to learn the mapping function due to its extreme nonlinearity.²⁸ We simulate 200 points based on appendix C of Lang and Witbrock,²⁸ and randomly select 80% of them for training and the remaining 20% for testing.

To apply FNN to this problem, we first conduct cross-validation based on accuracy in the testing dataset to select proper hyperparameters as introduced in Section 2.4: the number of hidden layers n_l , the number of nodes per layer n_w , the learning rate r_l , the batch size n_b , the number of epochs n_e , the dropout rate r_d and the activation function for hidden layers g_h . The above hyperparameters are for illustrative purposes. Based on the problem at hand, one can incorporate other parameters from different regulation approaches or training algorithms. Since there are seven hyperparameters involved, it is usually challenging to systematically and simultaneously fine-tune all parameters to their optimal values with a reasonable computational resource. Our approach is to first evaluate some hyperparameters separately while holding others as constants in a base model. We further utilize a random search type method² by randomly generating multiple sets of parameters around their suboptimal values or within certain supports, and then select the best set based on testing performance. There are some other approaches to select hyperparameters as discussed in Section 11.4 of Goodfellow et al.,² such as the grid search method, model-based hyperparameter optimization. The base or working FNN model has $n_l = 3$, $n_w = 30$, $r_l = 10^{-2}$, $n_e = 10^4$, $r_d = 0.2$, $n_b = 32$, and $g_h = \text{sigmoid}$. The activation function for the last layer is selected as sigmoid function for this binary classification problem.

In Table 1A, we evaluate six different FNN structures with varying n_l 's and varying n_w 's with other parameters specified by the base model. Among those six candidates, FNN with $n_l = 4$ and $n_w = 40$ has the highest accuracy of 90.0% in the testing dataset. Therefore, in the following random search method, we consider the range of integer n_l as $[3, 5]$ and integer $n_w \in [30, 50]$ around their corresponding suboptimal values. Similarly, $r_l = 0.01$ has the best testing performance in Table 1B, and $n_e = 10^4$ has the highest testing accuracy in Table 1C. Their supports are chosen as $r_l \in [0.001, 0.05]$ and integer $n_e \in [1000, 100\ 000]$, respectively. The activation function for hidden layers is chosen as the sigmoid function based on results in Table 1D. Next, we randomly simulate $B = 30$ sets of hyperparameters from their supports as listed in Table 1E. The range of r_d is set as $[0, 0.5]$ as discussed in Section 2.4.5 and the range $[10, 80]$ of integer n_b is a reasonable choice relative to the training dataset size of 160. They can also be decided by suboptimal values with additional simulations. Among $B = 30$ sets of parameters, the set as listed in Table 1E has the highest validation accuracy at 100% and is utilized for FNN training in this classification problem. For more complicated problems and more computational resources, one can also increase B in the random search method to find a better FNN if possible.

We label the testing dataset with black dots or triangles and the training dataset with symbols in Figure 3B and 3C. With inputs of x_1 and x_2 , a method predicts a triangle in the shaded area, and a dot in the unshaded area. The generalized linear model (GLM) with a threshold of 0.5 cannot properly classify two types of points (Figure 3B). On the other hand, FNN has a perfect prediction accuracy as visualized by all triangles being covered by shadow while all circles are not (Figure 3C).

An R markdown file and R code to replicate results in this section are available on Github with link provided in the Section "Data Availability Statement." The training of FNN is based on R packages `keras`²⁹ and `tensorflow`.³⁰

4 | SOME VARIANTS OF NEURAL NETWORKS

In this section, we discuss several more advanced frameworks based on FNN: convolutional neural networks (CNN) to process data with grid structure, recurrent neural networks (RNN) to handle sequence data, autoencoders to find an alternative representation of input data and generative adversarial networks (GAN) as generative models.

4.1 | Convolutional neural networks

Convolutional neural networks (CNN) is a specialized category of deep neural networks for processing data with grid-like topology, for example, image data.^{2,31} Let us consider a classification task of digits recognition based on a handwritten image. As shown in Figure 4, a standard CNN iteratively stacks convolutional layers and pooling layers followed by fully connected layers, which further connect to the classification output. In C1, a convolution operation is conducted on data from the input image and a certain learnable kernel to generate output referred to the feature map.² Each of the six feature

TABLE 1 Cross-validation based on accuracy in testing dataset to select feasible values of hyperparameters in FNN. Performance of FNN in training and testing datasets is summarized by varying FNN structures in (A), varying learning rates in (B), varying number of epochs in (C), and different activation functions for hidden layers in (D). Among 30 sets of randomly simulated parameters, values in the best set are summarized in (E)

(A) Varying FNN structures			
Number of hidden layers	Number of nodes per layer	Training accuracy	Testing accuracy
1	20	75.6%	42.5%
1	40	88.1%	70.0%
4	20	85.6%	85.0%
4	40	98.8%	90.0%
7	20	77.5%	70.0%
7	40	51.2%	45.0%
(B) Varying learning rates			
Learning rate	Training accuracy	Testing accuracy	
0.001	81.2%	72.5%	
0.01	96.2%	95.0%	
0.05	93.1%	87.5%	
0.1	90.6%	77.5%	
(C) Varying number of epochs			
Number of epochs	Training accuracy	Testing accuracy	
100	53.1%	47.5%	
1000	80.6%	75.0%	
10 000	96.2%	95.0%	
100 000	94.4%	92.5%	
(D) Different activation function for hidden layers			
Activation function for hidden layers	Training accuracy	Testing accuracy	
sigmoid	96.2%	95.0%	
ReLU	61.9%	55.0%	
tanh	93.1%	82.5%	
(E) Random search method to select a feasible set of hyperparameters			
	Support of random search	Selected value of hyperparameter	
Number of layers	[3, 5]	4	
Number of nodes per layer	[30, 50]	31	
Learning rate	[0.001, 0.05]	4.77×10^{-2}	
Number of epochs	[1000, 100 000]	7.05×10^4	
Dropout rate	[0, 0.5]	9.76×10^{-2}	
Batch size	[10, 80]	55	

Note: The set of parameter(s) with the highest testing accuracy in each subtable is highlighted in bold.

maps has its own kernel, such that similar characteristics of image can be extracted to the same feature map. Next, the pooling layer P2 utilizes a pooling function to replace the output from the previous layer at a certain location with a summary statistic of nearby outputs.² Maximum pooling³² and average pooling³³ are common operations in practice. The outputs from P2 are further transformed by convolution in C3 and pooling in P4. The network then connects to F5 as a fully connected layer that further links to the classification output, similar to the architecture of FNN shown in Figure 1. A toy example of convolution and pooling is provided in Figure 5.

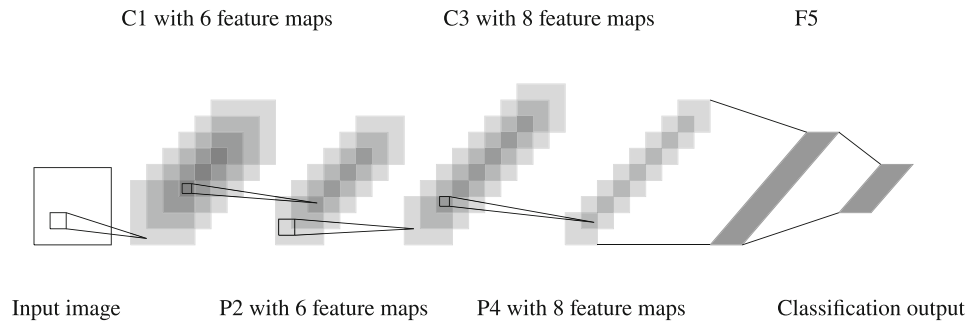


FIGURE 4 A CNN based on the architecture of LeNet-5 for image recognition presented in LeCun et al.³⁴ “Cx” denotes a convolutional layer, and “Px” denotes a pooling layer, and “Fx” stands for a fully-connected layer

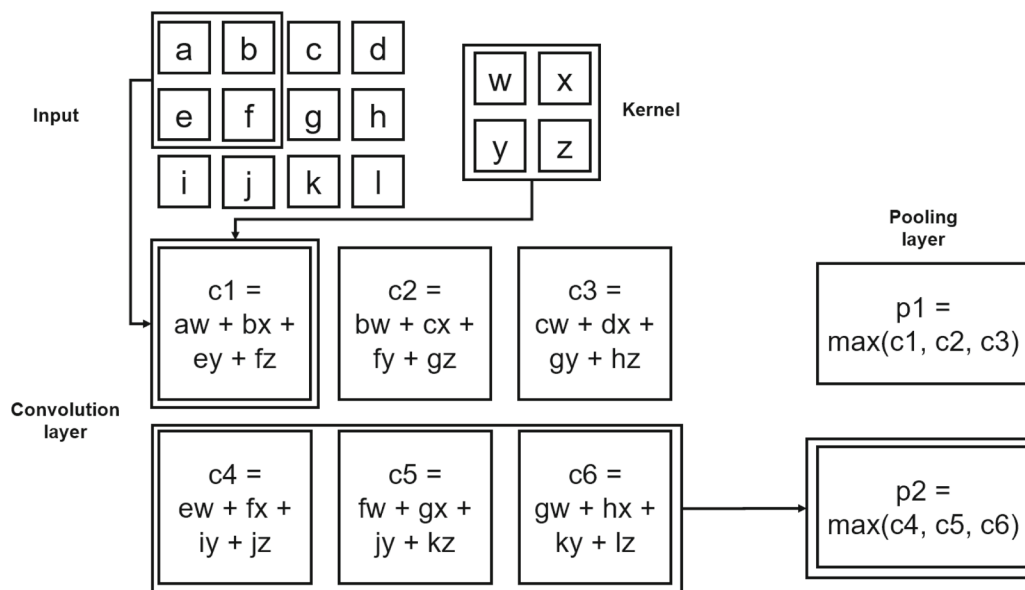


FIGURE 5 A toy demonstration of convolution and pooling of CNN based on Goodfellow et al.² and LeCun et al.³⁴ The input 3×4 matrix is applied with a 2×2 kernel matrix to obtain a 2×3 matrix for convolution layer. Maximum pooling is further conducted with a 1×3 stride matrix to get a 2×1 matrix for the pooling layer

As compared with a direct adoption of FNN with image input reshaped as a vector, there are several ideas behind CNN to make it suitable for image processing.² The first one is called sparse interactions and makes the dimension of kernel smaller than the input to detect small and meaningful features. On the contrary, every output unit connects and interacts with every input unit in FNN. Moreover, CNN has a parameter sharing property, as the same kernel is used for every input within the same feature map, but each parameter in the weight matrix of FNN is used only once. This feature not only reduces run time by training less parameters, but also potentially contributes to increased generalizability. Additionally, pooling makes CNN more robust to changes of input. This better fits the typical goal of CNN to detect whether a feature presents but not its exact location.

4.2 | Recurrent neural networks

Recurrent neural networks (RNN) are a family of neural networks to process sequential data,^{2,35} for example, data collected over time. Figure 6 shows a typical setting where both input $x^{(t)}$ and output $y^{(t)}$ are collected sequentially and are included in the model. The input $x^{(t)}$ is transformed by a weighting matrix U to a hidden layer $h^{(t)}$, which connects to intermediate output $o^{(t)}$ by V . With certain activation functions, $o^{(t)}$ is further mapped to approximate the training

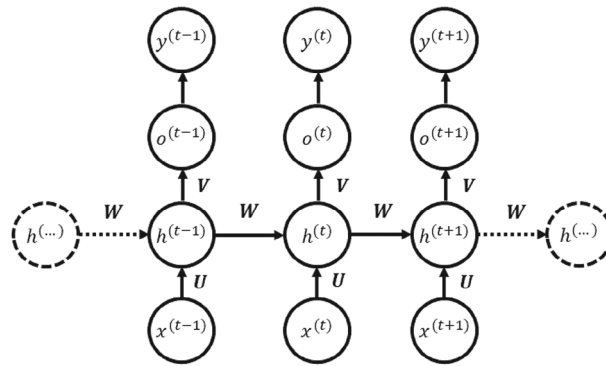


FIGURE 6 Recurrent neural networks (RNN) illustrated in Goodfellow et al²

target $y^{(t)}$. There are also some variants of RNN; for example, the recurrence is from intermediate output to the next hidden layer to enable parallel computing.²

Attention is a useful mechanism in RNN to focus on specific parts of the input sequence at each time.^{2,36} In an example of text translation, an attention-based system reads raw data of words in the source sentence and stores outputs of the reader containing a sequence of facts. Then it will sequentially perform this translation task by focusing on the content of one or a few output element(s).² This is similar to a technique in human translation where one first reads the whole sentence to get the general context and then produces translated words one at a time.² Attention can also be applied to other deep learning architecture, but not necessarily limited to RNN.

With the ability to handle sequential data, RNN has many successful applications such as speech recognition to process acoustic signal,³⁷ natural language processing (NLP),^{38,39} et cetera. Similar to CNN, the idea of parameter sharing is also important for RNN, as the same U , V , and W are utilized across the sequence in Figure 6. This property fits common text processing objectives to extract relevant information instead of specifically modeling exact locations of certain wordings.²

4.3 | Autoencoders

An autoencoder is a neural network that is trained to find an alternative representation of input data.² This technique has been widely applied to dimensionality reduction and information retrieval tasks.² It usually has an encoder function $\mathbf{h} = f(\mathbf{x})$ and a decoder function $\mathbf{r} = g(\mathbf{h})$. To prevent an autoencoder from trivially setting $g(f(\mathbf{x})) = \mathbf{x}$ and obtaining $g \circ f$ as the identity function, some constraints can be applied to retrieve important and representative properties of input data. For instance, an undercomplete autoencoder constrains $\mathbf{h}$ to have smaller dimension than $\mathbf{x}$ with the aim of capturing salient features of the training data.² Sparse autoencoders add a penalty term on $\mathbf{h}$ to the loss function without directly limiting the model capacity.⁴⁰ Denoising autoencoders use corrupted data $\tilde{\mathbf{x}}$ with noise as input and the original data $\mathbf{x}$ as output to learn good internal representation beyond identity function.⁴¹ Contractive autoencoders add a regularization term to the encoder $\mathbf{h}$ to shrink derivatives of f .⁴²

4.4 | Generative adversarial networks

The goal of generative adversarial networks (GAN) is to estimate a generator's distribution p_g over data $\mathbf{x}$ based on an adversarial process.⁴³ GAN has shown its advantages in many tasks:⁴⁴ predict a sharper and more recognizable next frame of a video,⁴⁵ or recover a better high-resolution image based on a downsampled low-resolution image.⁴⁶ GAN simultaneously trains two models: a generative network G to produce samples $\mathbf{x}$ based on noise input $\mathbf{z}$ by $\mathbf{x} = G(\mathbf{z}; \theta_g)$, and a discriminator network D to distinguish training data and samples from the generator G with $D(\mathbf{x}; \theta_d)$ indicating the probability that $\mathbf{x}$ is a real training example rather than a fake sample from G .² In the training of GAN, G and D play a two-player minimax game, where D maximizes its probability of assigning correct labels to both training examples $\mathbf{x}$ and examples $\mathbf{z}$ from G , while G attempts to fool the classifier D by mimicking the generative distribution p_g of $\mathbf{x}$.^{2,43} Specifically, G^* from

GAN is given by

$$G^* = \underset{G}{\operatorname{argmin}} \underset{D}{\operatorname{max}} V(G, D),$$

where $V(G, D)$ is a value or payoff function of the discriminator D . Ideally at convergence, G recovers the unknown p_g and D outputs 1/2 everywhere. However, GAN can be underfitted due to non-convergence issues in practice.² Some mitigation approaches have been proposed, for example an alternative formulation of the value function,⁴⁷ or a reformulation of the objective function.⁴³

5 | APPLICATIONS IN BIOMEDICAL FIELDS

In this section, we highlight some selective applications of deep learning techniques in biomedical problems.

5.1 | Medical image analyses

Computer vision has been one of the most active research areas for deep learning applications.² For medical images, deep learning techniques, especially CNN, can assist clinical diagnosis/detection, investigate patterns of disease progression, detect anatomical structures with image segmentation, align multiple images collected at different times, perform content-based image retrieval, et cetera.^{48,49} CNN was applied to classify skin cancer,³ and to detect diabetic retinopathy and diabetic macular edema⁴ with a level of competence comparable to clinicians. U-net was proposed based on convolutional architectures to segment neuronal structures in electron microscopic stacks.⁵ A spatial clockwork RNN was developed to segment the perimysium in hematoxylin and eosin staining skeletal muscle microscopy images.⁵⁰ CNN regressors were employed to directly estimate the transformation parameters in registration or alignment of medical images.⁵¹ A neural network-based image-on-scalar regression model was also proposed to investigate spatial patterns and covariate effects on functional magnetic resonance imaging (fMRI) of the brain.⁵²

5.2 | Complex innovative clinical trial designs

To evaluate efficacy and safety of study drugs more efficiently and more ethically, complex innovative clinical trial designs beyond traditional randomized controlled trials are receiving more interest.⁵³ Such innovative designs include but are not limited to historical data borrowing,⁵⁴ Sequential multiple assignment randomized trials (SMART),⁵⁵ adaptive clinical trials,⁵⁶ and master protocols.⁵⁷ More advanced approaches are usually needed to analyze data from those complex designs, and deep learning techniques have been applied and extended in some problems.

For example, a deep learning based outcome weighted learning algorithm was proposed to estimate individualized optimal combination therapy.⁵⁸ For this multilabel classification problem, the proposed method has a better performance in terms of treatment benefit than a naive implementation of label powerset (LP) for multiclass classification. FNN was integrated to optimization methods to identify a more powerful graphical approach for multiplicity control.⁵⁹ This deep learning enhanced optimization framework shows a better balance between robustness and time efficiency than some existing derivative-free constrained optimization algorithms. In the application of adaptive clinical trials, deep learning constructed statistics showed a higher power to detect potential treatment effect than several existing methods, and could essentially make clinical trials more efficient and ethical.⁶⁰ As an extension to multiple comparisons, a deep historical borrowing framework was developed to simultaneously and prospectively synthesize control information from multiple endpoints to ensure study integrity.⁶¹ A deep learning guided ensemble learning framework was proposed to construct a more accurate estimator of treatment effect in adaptive designs.⁶²

5.3 | Omics studies

Omics is the generic name of a research area covering genomic sequencing, gene expression, protein structure prediction and its interaction with other proteins or drugs.⁶³ Deep learning is also playing an important and active role in this area,^{64,65} with some selected works summarized below.

A deep learning method (D-GEX) was introduced to infer the expression of target genes from the expression of landmark genes with a better performance of mean absolute error than the linear regression based approach adopted in the NIH integrated network-based cellular signatures (LINCS) program.⁶⁶ The DeepBind was developed based on CNN to understand the sequence specificities of DNA- and RNA-binding proteins.⁶⁷ This method outperforms some state-of-the-art methods based on several evaluation metrics, even when training on in vitro data and testing on in vivo data. DANN as a deep learning approach was proposed to annotate the pathogenicity of genetic variants that outperformed the support vector machine with linear kernel.⁶⁸ DNSS with deep learning network architectures was trained to predict secondary structures of protein with satisfactory performance based on accuracy.⁶⁹

6 | CONCLUDING REMARKS

In this article, we provide a general introduction to deep learning and highlight some of its biomedical applications. However, this is by no means to recommend that deep learning is the “go-to method” for every problem. There are many other powerful machine learning techniques, such as XGBoost⁷⁰ as a gradient boosting approach. It is usually preferred to evaluate and test several different methods to find the best one for a particular problem. Additionally, multiple techniques can also be integrated to find a better solution for complicated real-world problems. For example, AlphaGo combined deep learning and advanced tree search methods from reinforcement learning and became the first program to defeat a world champion in the game of Go.⁷¹

Deep learning is a fast growing field with many ongoing works. Some key questions have been partially answered or need further investigation. For example, interpretation is an essential component to address biomedical problems, and interpretable neural networks are receiving more attention. An interpretable CNN is proposed with each filter in a convolution layer representing a specific object part.⁷² A unified framework SHAP (shapley additive explanations) for interpreting predictions was developed to assign each feature an importance value for a particular prediction.⁷³ In the application of medical image classification, layerwise relevance propagation (LRP) transformed neural networks decisions into heatmaps indicating each data point's relevance for the outcome of the decision.⁷⁴ Deep learning can be naturally applied to a high-dimension setting to perform variable selection and related tasks, but a full and complete understanding of corresponding statistical properties is still an active research area.^{75,76} On the one hand, deep learning is an extremely automatic and powerful framework to achieve impressive performance in many tasks. On the other hand, it then becomes more challenging to quantitatively understand certain properties of this state-of-the-art method with potential extensions. Another interesting research area is to balance the trade-off between these two aspects by identifying tailored or customized neural networks to address a class of problems but with statistical properties and operating characteristics that are relatively easier to characterize.

ACKNOWLEDGEMENTS

The author would like to thank the Editor, an anonymous Associate Editor and two anonymous reviewers for their constructive comments, which significantly improved this article. Tianyu Zhan is an employee of AbbVie Inc. This publication was neither originated nor managed by AbbVie, and it does not communicate results of AbbVie-sponsored Scientific Research. Thus, it is not in scope of the AbbVie Publication Procedure (PUB-100)

DATA AVAILABILITY STATEMENT

Data sharing is not applicable to this article because results are generated by computer simulations. The R markdown file and the R code to reproduce results in Section 3 are available at https://github.com/tian-yu-zhan/DNN_tutorial.

ORCID

Tianyu Zhan  <https://orcid.org/0000-0002-8572-4539>

REFERENCES

1. Chollet F, Allaire JJ. *Deep Learning with R*. Shelter Island, NY: Manning Publications; 2018.
2. Goodfellow I, Bengio Y, Courville A, Bengio Y. *Deep Learning*. Cambridge, MA: MIT Press; 2016.
3. Esteva A, Kuprel B, Novoa RA, et al. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*. 2017;542(7639):115-118.

4. Gulshan V, Peng L, Coram M, et al. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *JAMA*. 2016;316(22):2402-2410.
5. Ronneberger O, Fischer P, Brox T. U-Net: Convolutional networks for biomedical image segmentation. Proceedings of the International Conference on Medical Image Computing and Computer-assisted Intervention. 2015;234-241.
6. Goodfellow IJ, Bulatov Y, Ibarz J, Arnoud S, Shet V. Multi-digit number recognition from street view imagery using deep convolutional neural networks. arXiv preprint, arXiv:1312.6082 2013.
7. Jaitly N, Hinton G. Learning a better representation of speech soundwaves using restricted Boltzmann machines. Proceedings of the 2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). 2011;5884-5887.
8. Mohamed Ar, Sainath TN, Dahl G, Ramabhadran B, Hinton GE, Picheny MA. Deep belief networks using discriminative features for phone recognition. Proceedings of the 2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). 2011;5060-5063.
9. Dahl GE, Yu D, Deng L, Acero A. Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition. *IEEE Trans Audio Speech Lang Process*. 2011;20(1):30-42.
10. Bengio Y, Ducharme R, Vincent P, Janvin C. A neural probabilistic language model. *J Machine Learn Res*. 2003;3:1137-1155.
11. Morin F, Bengio Y. Hierarchical probabilistic neural network language model. *Int Workshop Artificial Intell Stat*. 2005;246-252.
12. Mnih A, Teh YW. A fast and simple algorithm for training neural probabilistic language models. Proceedings of the 29th International Conference on International Conference on Machine Learning. 2012;419-426.
13. Pomerleau DA. ALVINN: An autonomous land vehicle in a neural network. *Adv Neural Info Process Syst*. 1988;305-313.
14. Muller U, Ben J, Cosatto E, Flepp B, Cun Y. Off-road obstacle avoidance through end-to-end learning. *Adv Neural Info Process Syst*. 2005;739-746.
15. Bojarski M, Yeres P, Choromanska A, et al. Explaining how a deep neural network trained with end-to-end learning steers a car. arXiv preprint, arXiv:1704.07911 2017.
16. Jarrett K, Kavukcuoglu K, Ranzato M, LeCun Y. What is the best multi-stage architecture for object recognition? Proceedings of the IEEE 12th International Conference on Computer Vision. 2009;2146-2153.
17. Nair V, Hinton GE. Rectified linear units improve restricted Boltzmann machines. *Int Conf Machine Learn*. 2010;807-814.
18. Cybenko G. Approximation by superpositions of a sigmoidal function. *Math Control Signals Syst*. 1989;2(4):303-314.
19. Boser BE, Guyon IM, Vapnik VN. A training algorithm for optimal margin classifiers. Proceedings of the 5th Annual ACM Workshop on Computational Learning Theory. 1992;144-152.
20. Duchi J, Hazan E, Singer Y. Adaptive subgradient methods for online learning and stochastic optimization. *J Machine Learning Res*. 2011;12(7):2121-2159.
21. Hinton G, Srivastava N, Swersky K. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. *Coursera*. 2012.
22. Kingma DP, Ba J. ADAM: A method for stochastic optimization. arXiv preprint, arXiv:1412.6980.
23. Hornik K, Stinchcombe M, White H. Multilayer feedforward networks are universal approximators. *Neural Networks*. 1989;2(5):359-366.
24. Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R. Dropout: a simple way to prevent neural networks from overfitting. *J Machine Learn Res*. 2014;15(1):1929-1958.
25. Bishop CM. Regularization and complexity control in feed-forward networks. *Int Conf Artificial Neural Networks*. 1995;1:141-148.
26. Lasserre JA, Bishop CM, Minka TP. Principled hybrids of generative and discriminative models. Proceedings of the 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. 2006;1:87-94.
27. Breiman L. Bagging predictors. *Machine Learn*. 1996;24(2):123-140.
28. Lang KJ, Witbrock MJ. Learning to tell two spirals apart. Proceedings of the 1988 Connectionist Models Summer School. 1988; 52-59.
29. Allaire J, Chollet F. keras: R Interface to 'Keras'. R package version 2.6.0.9001. 2021.
30. Allaire J, Tang Y. tensorflow: R Interface to 'TensorFlow'. R package version 2.6.0.9000. 2021.
31. LeCun Y. Generalization and network design strategies. Technical Report CRG-TR-89-4, University of Toronto; 1989.
32. Zhou YT, Chellappa R. Computation of optical flow using a neural network. Proceedings of the IEEE 1988 International Conference on Neural Networks. 1988;71-78.
33. LeCun Y, Boser B, Denker J, et al. Handwritten digit recognition with a back-propagation network. *Adv Neural Info Process Syst*. 1989;2:396-404.
34. LeCun Y, Bottou L, Bengio Y, Haffner P. Gradient-based learning applied to document recognition. *Proc IEEE*. 1998;86(11):2278-2324.
35. Rumelhart DE, Hinton GE, Williams RJ. Learning representations by back-propagating errors. *Nature*. 1986;323(6088):533-536.
36. Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. arXiv preprint, arXiv:1409.0473.
37. Graves A. Generating sequences with recurrent neural networks. arXiv preprint, arXiv:1308.0850.
38. Kalchbrenner N, Blunsom P. Recurrent continuous translation models. Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing. 2013;1700-1709.
39. Cho K, Merriënboer vB, Gulcehre C, Bougares F, Schwenk H, Bengio Y. Learning phrase representations using RNN encoder-decoder for statistical machine translation. Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP 2014). 2014.
40. Ranzato M, Poultney C, Chopra S, Cun Y. Efficient learning of sparse representations with an energy-based model. *Adv Neural Info Process Syst*. 2006;19:1137-1144.
41. Gallinari P, LeCun Y, Thiria S, Fogelman-Soulie F. Memoires associatives distribuees. Proceedings of COGNITIVA. 1987;87:93.

42. Salah R, Vincent P, Muller X, et al.. Contractive auto-encoders: Explicit invariance during feature extraction. *Proceedings of the International Conference on Machine Learning*. 2011:833-840.
43. Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets. *Adv Neural Info Process Syst*. 2014;27:2672-2680.
44. Goodfellow I. NIPS 2016 tutorial: Generative adversarial networks. arXiv preprint, arXiv:1701.00160.
45. Lotter W, Kreiman G, Cox D. Unsupervised learning of visual structure using predictive generative networks. arXiv preprint, arXiv:1511.06380.
46. Ledig C, Theis L, Huszár F, et al. Photo-realistic single image super-resolution using a generative adversarial network. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017:4681-4690.
47. Goodfellow IJ. On distinguishability criteria for estimating generative models. arXiv preprint, arXiv:1412.6515.
48. Litjens G, Kooi T, Bejnordi BE, et al. A survey on deep learning in medical image analysis. *Medical Image Anal*. 2017;42:60-88.
49. Liu X, Gao K, Liu B, et al. Advances in deep learning-based medical image analysis. *Health Data Sci*. 2021;2021.
50. Xie Y, Zhang Z, Sapkota M, Yang L. Spatial clockwork recurrent neural network for muscle perimysium segmentation. *Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention*. 2016:185-193.
51. Miao S, Wang ZJ, Liao R. A CNN regression approach for real-time 2D/3D registration. *IEEE Trans Med Imaging*. 2016;35(5):1352-1363.
52. Zhang D, Li L, Sripada C, Kang J. Image-on-Scalar regression via deep neural networks. arXiv preprint, arXiv:2006.09911.
53. Food and Drug Administration. Interacting with the FDA on complex innovative clinical trial designs for drugs and biological products. 2020. <https://www.fda.gov/regulatory-information/search-fda-guidance-documents/interacting-fda-complex-innovative-trial-designs-drugs-and-biological-products>.
54. Schmidli H, Gsteiger S, Roychoudhury S, O'Hagan A, Spiegelhalter D, Neuenschwander B. Robust meta-analytic-predictive priors in clinical trials with historical control information. *Biometrics*. 2014;70(4):1023-1032.
55. Murphy SA. An experimental design for the development of adaptive treatment strategies. *Stat Med*. 2005;24(10):1455-1481.
56. Bretz F, Koenig F, Brannath W, Glimm E, Posch M. Adaptive designs for confirmatory clinical trials. *Stat Med*. 2009;28(8):1181-1217.
57. Woodcock J, LaVange LM. Master protocols to study multiple therapies, multiple diseases, or both. *New En J Med*. 2017;377(1):62-70.
58. Liang M, Ye T, Fu H. Estimating individualized optimal combination therapies through outcome weighted deep learning algorithms. *Stat Med*. 2018;37(27):3869-3886.
59. Zhan T, Hartford A, Kang J, Offen W. Optimizing graphical procedures for multiplicity control in a confirmatory clinical trial via deep learning. *Stat Biopharm Res*. 2022;14(1):92-102.
60. Zhan T, Kang J. Finite-sample two-group composite hypothesis testing via machine learning. *J Comput Graph Stat*. 2022;1-30.
61. Zhan T, Zhou Y, Geng Z, et al. Deep historical borrowing framework to prospectively and simultaneously synthesize control information in confirmatory clinical trials with multiple endpoints. *J Biopharm Stat*. 2022;32(1):90-106.
62. Zhan T, Fu H, Kang J. Deep neural networks guided ensemble learning for point estimation. arXiv preprint, arXiv:2105.06523.
63. Zemouri R, Zerhouni N, Racoceanu D. Deep learning in the biomedical applications: Recent and future status. *Appl Sci*. 2019;9(8):1526.
64. Mamoshina P, Vieira A, Putin E, Zhavoronkov A. Applications of deep learning in biomedicine. *Mol Pharm*. 2016;13(5):1445-1454.
65. Cao C, Liu F, Tan H, et al. Deep learning and its applications in biomedicine. *Genomics Proteomics Bioinfo*. 2018;16(1):17-32.
66. Chen Y, Li Y, Narayan R, Subramanian A, Xie X. Gene expression inference with deep learning. *Bioinformatics*. 2016;32(12):1832-1839.
67. Alipanahi B, Delong A, Weirauch MT, Frey BJ. Predicting the sequence specificities of DNA-and RNA-binding proteins by deep learning. *Nature Biotechnol*. 2015;33(8):831-838.
68. Quang D, Chen Y, Xie X. DANN: a deep learning approach for annotating the pathogenicity of genetic variants. *Bioinformatics*. 2015;31(5):761-763.
69. Spencer M, Eickholt J, Cheng J. A deep learning network approach to *ab initio* protein secondary structure prediction. *IEEE/ACM Trans Comput Biology Bioinfo*. 2014;12(1):103-112.
70. Chen T, Guestrin C. XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016;785-794.
71. Silver D, Schrittwieser J, Simonyan K, et al. Mastering the game of go without human knowledge. *Nature*. 2017;550(7676):354-359.
72. Zhang Q, Wu YN, Zhu SC. Interpretable convolutional neural networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018;8827-8836.
73. Lundberg SM, Lee SI. A unified approach to interpreting model predictions. *Adv Neural Info Process Syst*. 2017;30:4765-4774.
74. Sturm I, Lapuschkin S, Samek W, Müller KR. Interpretable deep neural networks for single-trial EEG classification. *J Neurosci Methods*. 2016;274:141-145.
75. Liang F, Li Q, Zhou L. Bayesian neural networks for selection of drug sensitive genes. *J Am Stat Association*. 2018;113(523):955-972.
76. Chen Y, Gao Q, Liang F, Wang X. Nonlinear variable selection via deep neural networks. *J Comput Graph Stat*. 2021;30(2):484-492.

How to cite this article: Zhan T. DL 101: Basic introduction to deep learning with its application in biomedical related fields. *Statistics in Medicine*. 2022;41(26):5365-5378. doi: 10.1002/sim.9564